

Data types in R

Course so far

So far, we know how to do key wrangling tasks in R:

- Grouping, summarizing, mutating
- Pivoting and joining

What is next: Data wrangling in Python!

To get there:

- How does R store data?
- Similarities and differences between R and Python objects for data storage and wrangling

Atomic vectors

So far, we have worked with rectangular data (data tables which are collections of columns):

	name	height	species	mass	hair_color
1	Luke Skywalker	172	Human	77	blond
2	C-3P0	167	Droid	75	<NA>
3	R2-D2	96	Droid	32	<NA>
4	Darth Vader	202	Human	136	none
5	Leia Organa	150	Human	49	brown
6	Owen Lars	178	Human	120	brown, grey

Atomic vectors

So far, we have worked with rectangular data (data tables which are collections of columns):

	name	height	species	mass	hair_color
1	Luke Skywalker	172	Human	77	blond
2	C-3P0	167	Droid	75	<NA>
3	R2-D2	96	Droid	32	<NA>
4	Darth Vader	202	Human	136	none
5	Leia Organa	150	Human	49	brown
6	Owen Lars	178	Human	120	brown, grey

Each column is an **atomic vector**:

```
1 starwars$name
```

```
[1] "Luke Skywalker"      "C-3P0"               "R2-D2"
[4] "Darth Vader"         "Leia Organa"         "Owen Lars"
[7] "Beru Whitesun Lars"  "R5-D4"               "Biggs
Darklighter"
[10] "Obi-Wan Kenobi"      "Anakin Skywalker"    "Wilhuff Tarkin"
```

[13]	"Chewbacca"	"Han Solo"	"Greedo"
[16]	"Jabba Desilijic Tiure"	"Wedge Antilles"	"Jek Tono Porkins"
[19]	"Yoda"	"Palpatine"	"Boba Fett"
[22]	"IG-88"	"Bossk"	"Lando Calrissian"
[25]	"Lobot"	"Ackbar"	"Mon Mothma"
[28]	"Arvel Crynyd"	"Wicket Systri Warrick"	"Nien Nunb"
[31]	"Qui-Gon Jinn"	"Nute Gunray"	"Finis Valorum"

Atomic vectors

Atomic vectors are one-dimensional objects, and are indexed with `[ ]`:

```
1 x <- c(0, 23, 98)
2 length(x)
```

```
[1] 3
```

```
1 x[1]
```

```
[1] 0
```

```
1 x[2]
```

```
[1] 23
```

```
1 x[1:2]
```

```
[1] 0 23
```

```
1 x[c(1, 3)]
```

```
[1] 0 98
```

Atomic vectors

Vectors can contain numbers, booleans, characters, etc:

```
1 x <- c(0, 1, 2)
2 x
```

```
[1] 0 1 2
```

```
1 typeof(x)
```

```
[1] "double"
```

```
1 x <- c("a", "b", "c")
2 x
```

```
[1] "a" "b" "c"
```

```
1 typeof(x)
```

```
[1] "character"
```

The `typeof` function tells what *type* of object we have

Vectors of multiple types?

```
1 x <- c(0, 1, "a")  
2 x  
3 x[1] + 1
```

What do you think will happen when we run this code?

Vectors of multiple types?

```
1 x <- c(0, 1, "a")  
2 x
```

```
[1] "0" "1" "a"
```

```
1 x[1] + 1
```

```
Error in x[1] + 1: non-numeric argument to binary operator
```

Vectors of multiple types?

```
1 x <- c(0, 1, "a")  
2 x
```

```
[1] "0" "1" "a"
```

```
1 x[1] + 1
```

```
Error in x[1] + 1: non-numeric argument to binary operator
```

Atomic vectors only contain one type.

Lists

```
1 x <- list(c(0, 1), "a")  
2 x
```

```
[[1]]
```

```
[1] 0 1
```

```
[[2]]
```

```
[1] "a"
```

Lists

```
1 x <- list(c(0, 1), "a")  
2 x
```

```
[[1]]  
[1] 0 1
```

```
[[2]]  
[1] "a"
```

```
1 x[[1]]
```

```
[1] 0 1
```

```
1 x[[1]][1]
```

```
[1] 0
```

Lists

```
1 x <- list(c(0, 1), "a")  
2 x
```

```
[[1]]  
[1] 0 1
```

```
[[2]]  
[1] "a"
```

```
1 x[[1]]
```

```
[1] 0 1
```

```
1 x[[1]][1]
```

```
[1] 0
```

```
1 typeof(x[[1]])
```

```
[1] "double"
```

```
1 x[[2]]
```

```
[1] "a"
```

```
1 typeof(x[[2]])
```

Visualizing list structure

```
1 x1 <- list(c(1, 2), c(3, 4))  
2 x1
```

```
[[1]]
```

```
[1] 1 2
```

```
[[2]]
```

```
[1] 3 4
```

```
1 x2 <- list(list(1, 2), list(3, 4))  
2 x2
```

```
[[1]]
```

```
[[1]][[1]]
```

```
[1] 1
```

```
[[1]][[2]]
```

```
[1] 2
```

```
[[2]]
```

```
[[2]][[1]]
```

[1] 3

۱۱۱۱ ۱۱۱۱

Indexing lists

```
1 x <- list(c(1, 2), c(3, 4))  
2  
3 x[1]
```

```
[[1]]
```

```
[1] 1 2
```

```
1 typeof(x[1])
```

```
[1] "list"
```

```
1 x[[1]]
```

```
[1] 1 2
```

```
1 typeof(x[[1]])
```

```
[1] "double"
```

- `x[1]` returns a *list* which contains the first component of `x`
- `x[[1]]` returns the object stored in the first component

Rectangular data

```
1 starwars
```

```
# A tibble: 87 × 14
  name      height  mass hair_color skin_color eye_color birth_year sex
gender
  <chr>      <int> <dbl> <chr>      <chr>      <chr>      <dbl>
<chr> <chr>
1 Luke Sk...   172    77 blond      fair        blue        19
male mascu...
2 C-3P0       167    75 <NA>      gold        yellow      112
none mascu...
3 R2-D2        96    32 <NA>      white, bl... red         33
none mascu...
4 Darth V...  202   136 none      white       yellow     41.9
male mascu...
```

Rectangular data in R

In R, there are two main ways of storing rectangular data:

- matrices
- data frames

Matrices

A *matrix* generalizes a vector to *two* dimensions:

```
1 x <- matrix(c(1, 2, 3, 4, 5, 6), nrow=2)
2 x
```

	[,1]	[,2]	[,3]
[1,]	1	3	5
[2,]	2	4	6

- Each row is an atomic vector
- Each column is an atomic vector

Indexing matrices

```
1 x <- matrix(c("a", "b", "c", "d", "e", "f"), nrow=2)
2 x
```

```
      [,1] [,2] [,3]
[1,] "a"  "c"  "e"
[2,] "b"  "d"  "f"
```

What will this code return?

```
1 x[2, 3]
```

Indexing matrices

```
1 x <- matrix(c("a", "b", "c", "d", "e", "f"), nrow=2)
2 x
```

```
      [,1] [,2] [,3]
[1,] "a"  "c"  "e"
[2,] "b"  "d"  "f"
```

What will this code return?

```
1 x[2, 3]
```

```
[1] "f"
```

Indexing matrices

```
1 x <- matrix(c("a", "b", "c", "d", "e", "f"), nrow=2)
2 x
```

	[,1]	[,2]	[,3]
[1,]	"a"	"c"	"e"
[2,]	"b"	"d"	"f"

What will this code return?

```
1 x[2,]
```

Indexing matrices

```
1 x <- matrix(c("a", "b", "c", "d", "e", "f"), nrow=2)
2 x
```

```
      [,1] [,2] [,3]
[1,] "a"  "c"  "e"
[2,] "b"  "d"  "f"
```

What will this code return?

```
1 x[2,]
```

```
[1] "b" "d" "f"
```

Indexing matrices

```
1 x <- matrix(c("a", "b", "c", "d", "e", "f"), nrow=2)
2 x
```

```
      [,1] [,2] [,3]
[1,] "a"  "c"  "e"
[2,] "b"  "d"  "f"
```

What will this code return?

```
1 x[,2]
```


Indexing matrices

```
1 x <- matrix(c("a", "b", "c", "d", "e", "f"), nrow=2)
2 x
```

```
      [,1] [,2] [,3]
[1,] "a"  "c"  "e"
[2,] "b"  "d"  "f"
```

What will this code return?

```
1 x[,2]
```

```
[1] "c" "d"
```

Uses and limitations of matrices

- Correspond to the matrices we know and love from linear algebra
- Usually the right way to store 2-d data for doing math (like matrix multiplication)
- Like atomic vectors, contain only one type of data

```
1 x <- matrix(c(1, 2, 3, 'a', 5, 6), nrow=2)
2 x
```

	[,1]	[,2]	[,3]
[1,]	"1"	"3"	"5"
[2,]	"2"	"a"	"6"

Data frames

```
1 example_df <- data.frame(x = c(1, 2, 3),  
2                           y = c('a', 'b', 'c'))  
3 example_df
```

	x	y
1	1	a
2	2	b
3	3	c

Aside: what *are* data frames?

```
1 example_df <- data.frame(x = c(1, 2, 3),  
2                           y = c('a', 'b', 'c'))  
3 typeof(example_df)
```

```
[1] "list"
```

- Matrices are like a 2-d atomic vector
- Data frames are a special type of list! With some requirements:
 - Each component is a vector
 - Each component has the same length

Indexing data frames

[] can work for indexing data frames, just like matrices:

```
1 example_df <- data.frame(x = c(1, 2, 3),  
2                           y = c('a', 'b', 'c'))  
3 example_df
```

```
  x y  
1 1 a  
2 2 b  
3 3 c
```

```
1 example_df[2, 1]
```

```
[1] 2
```

Indexing data frames

Like lists, `[[ ]]` and `$` can also be used:

```
1 example_df <- data.frame(x = c(1, 2, 3),  
2                           y = c('a', 'b', 'c'))  
3 example_df
```

```
  x y  
1 1 a  
2 2 b  
3 3 c
```

```
1 example_df$x
```

```
[1] 1 2 3
```

```
1 example_df[["x"]]
```

```
[1] 1 2 3
```

Class activity

- Work with a neighbor on the class activity
- At the end of class, submit your work as an HTML file on Canvas (one per group, list all your names)